ENSPY • GÉNIE INFORMATIQUE • GIF3025-2

Atelier Pratique OpenSSL

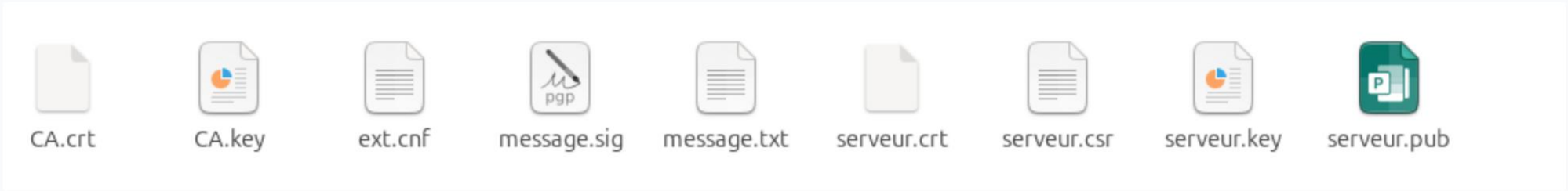
Démonstration concrète de création d'une mini-PKI, émission de certificats et vérification de signature numérique

Auteurs :
Groupe 30 (Nkouathio, Mbende, Moiffo, Kengne, Kadji)

Encadreur :
Dr. Hervé TALE KALACHI

RÉSULTAT DU RÉPERTOIRE

Artefacts Générés lors de l'Atelier



9 fichiers clés constituant la CA racines, la configuration du serveur, les certificats et les signatures.

PREUVE D'EXÉCUTION

Traces des Commandes Terminal OpenSSL

```
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl req -x509 -new -key CA.key -sha256 -days 3650 \
> -subj "/C=CM/ST=Centre/L=Yaounde/O=Ma CA/CN=exemple.local" \
> -out CA.crt
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl req -in serveur.csr -noout -verify
Certificate request self-signature verify OK
nkouathio@nkouathio-Latitude-5580:~/exposée$ cat > ext.cnf <<'EOF'
> basicConstraints = critical, CA:FALSE
> keyUsage = critical, digitalSignature
> extendedKeyUsage = serverAuth
> subjectAltName = DNS:www.mon-site.cm, DNS:localhost, IP:127.0.0.1
> EOF
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl x509 -req -in serveur.csr -CA CA.crt -CAkey CA.key \
> -CAcreateseruak -days 90 -sha256 -extfile ext.cnf \
> -out serveur.crt
Certificate request self-signature ok
subject=C = CM, ST = Centre, L = Yaounde, O = Ma Societe Sarl, OU = IT, CN = www.mon-site.cm, emailAddress = admin@mon-site.cm
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl verify -CAfile CA.crt serveur.crt
serveur.crt: OK
nkouathio@nkouathio-Latitude-5580:~/exposée$ echo "Bonjour Bob" > message.txt
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl dgst -sha256 -sign serveur.key -out message.sig message.txt
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl pkey -in serveur.key -pubout -out serveur.pub
nkouathio@nkouathio-Latitude-5580:~/exposée$ openssl dgst -sha256 -verify serveur.pub -signature message.sig message.txt
Verified OK
nkouathio@nkouathio-Latitude-5580:~/exposée$
```

ÉTAPE 1 & 2

Création de la CA Racine Auto-Signée

1. Génération Clé Privée CA

```
openssl genpkey -algorithm RSA -pkeyopt
rsa_keygen_bits:2048 -out CA.key
```

Produit le fichier d'autorité `CA.key`.

2. Certificat Racine Auto-Signé

```
openssl req -x509 -new -key CA.key -sha256 -days 3650 -out
CA.crt
```

Produit le certificat d'ancrage `CA.crt` (valide 10 ans).

ARTEFACT RÉEL

Contenu du Certificat CA Racine (CA.crt)

```
-----BEGIN CERTIFICATE-----
MIIDkTCCAnmgAwIBAgIUGJLt2XOaCVhbDXHf3o648UDEX+swDQYJKoZIhvcNAQEL
BQAwWDELMAkGA1UEBhMCQ00xDzANBgNVBAgMBkNlbnRyZTEQMA4GA1UEBwwHWWFv
dW5kZTEOMAwGA1UECgwFTWEgQ0ExFjAUBgNVBAMMDWV4ZW1wbGUubG9jYWwwHhcN
MjYxMTA5MTc1ODE1WhcNMzYxMTA2MTc1ODE1WjBYMQswCQYDVQQGEwJDTTEPMA0G
... [ Tronqué pour affichage ] ...
F2gorreyFipyqwVG3x3cRw6MBDimRZurJ/mhYz2clmyBaKvLI1vFIIcdITLv36tXA
-----END CERTIFICATE-----
```

ÉTAPE 3 & 4

Demande de Certificat Serveur (CSR)

Création de serveur.key & serveur.csr

```
openssl genpkey -algorithm RSA -out serveur.key
openssl req -new -key serveur.key -sha256 -out serveur.csr
```

Définit le Nom Distinctif (DN) : `CN=www.mon-site.cm`, `O=Ma Societe Sarl`, `C=CM`.

Vérification de la requête

```
openssl req -in serveur.csr -noout -verify
# Résultat : Certificate request self-signature verify OK
```

Atteste que la demande est valide et signée par sa clé privée.

ÉTAPE 5 & 6

Configuration des Extensions & Scellement

Fichier ext.cnf

```
basicConstraints = critical, CA:FALSE
keyUsage = critical, digitalSignature
extendedKeyUsage = serverAuth
subjectAltName = DNS:www.mon-site.cm, IP:127.0.0.1
```

Signature par la CA

```
openssl x509 -req -in serveur.csr -CA CA.crt -CAkey CA.key -
CAcreateserial -days 90 -sha256 -extfile ext.cnf -out
serveur.crt
```

Génère le certificat final `serveur.crt`.

ÉTAPE 7

Vérification de la Chaîne de Confiance

```
$ openssl verify -CAfile CA.crt serveur.crt
serveur.crt: OK
```

Le système confirme que le certificat serveur est valablement signé par l'autorité `CA.crt`.

ÉTAPE 8

Signature et Contrôle d'Intégrité de message.txt

1. Création & Signature

```
echo "Bonjour Bob" > message.txt
openssl dgst -sha256 -sign serveur.key -out message.sig
message.txt
```

Création de `message.txt` et extraction de la clé publique `serveur.pub`.

2. Vérification par Bob

```
openssl dgst -sha256 -verify serveur.pub -signature
message.sig message.txt
# Résultat : Verified OK
```

Garantit l'origine et l'inaltérabilité absolue du message.

BILAN DE L'ATELIER

Validation Pratique des Objectifs

PKI Opérationnelle

Création réussie d'une autorité de certification et émission de certificat serveur.

Signature Validée

Contrôle d'intégrité et de non-répudiation confirmé avec le retour `Verified OK`.

Merci pour votre attention • Fin de la Démonstration